

07 — PostgreSQL Indexes: Complete Reference

Master every PostgreSQL index type, design strategy, and maintenance technique.

Module Overview

This module is a complete technical reference for PostgreSQL indexing — from the fundamentals of why indexes exist through the internals of every index type to production maintenance and 60+ interview questions.

Estimated study time: 20–30 hours for complete mastery
Prerequisites: SQL basics, understanding of tables and queries
What you'll build: The ability to design, implement, and diagnose indexes for any PostgreSQL workload at any scale.

Contents

File	Topic	Difficulty	Lines
01_index_fundamentals.md	Why indexes exist, cost tradeoffs, planner decisions	Beginner	~450
02_btree_indexes.md	B-tree structure, pages, searches, range scans	Intermediate	~550
03_hash_indexes.md	Hash table internals, when to use, limitations	Intermediate	~400
04_gin_indexes.md	JSONB, arrays, full-text, pg_trgm	Intermediate	~550
05_gist_indexes.md	Geometry, ranges, exclusion constraints, PostGIS	Intermediate	~500
06_brin_indexes.md	Block ranges, correlation, time-series	Intermediate	~450
07_spgist_indexes.md	Quadrees, tries, non-overlapping partitions	Advanced	~450
08_partial_indexes.md	Predicate indexes, space savings, use cases	Intermediate	~500
09_composite_indexes.md	Column ordering, prefix matching, sort indexes	Intermediate	~500
10_covering_indexes.md	INCLUDE clause, index-only scans, visibility map	Intermediate	~500
11_expression_indexes.md	Functional indexes, case-insensitive, JSONB fields	Intermediate	~500
12_index_maintenance.md	Bloat, REINDEX, VACUUM, monitoring	Advanced	~550
13_index_interview_guide.md	60+ Q&A, whiteboard exercises, cheat sheet	All levels	~650

Learning Path

Path 1: Quick Start (4–6 hours)

Best for developers who need to improve query performance now:

1. [01 index fundamentals.md](#) — understand the why
2. [02 btree indexes.md](#) — master the default
3. [09 composite indexes.md](#) — column ordering rules
4. [10 covering indexes.md](#) — index-only scans
5. [08 partial indexes.md](#) — space savings

Path 2: Complete Mastery (20–30 hours)

Read all files in order (01 through 13).

Path 3: Interview Preparation (6–8 hours)

1. [01 index fundamentals.md](#) — fundamentals (required)
2. [02 btree indexes.md](#) — deep dive (required)
3. [08 partial indexes.md](#) — practical design
4. [09 composite indexes.md](#) — design strategy
5. [13 index interview guide.md](#) — practice all questions

Path 4: Specialized Type Deep Dive (8–10 hours)

1. [04 gin indexes.md](#) — JSONB and full-text
2. [05 gist indexes.md](#) — geometry and ranges
3. [06 brin indexes.md](#) — time-series and large tables
4. [07 spgist indexes.md](#) — spatial and hierarchical

Core Concepts at a Glance

Index Type Decision Tree

```
What type of data are you indexing?
|
|— Scalar (int, text, date, uuid, etc.)?
|   |— Need range queries / sorting? → B-tree (default)
|   |— Long keys, equality only? → Hash (consider)
|   |— Very large table, sequential? → BRIN
|
|— Multi-valued (arrays, JSONB, tsvector)?
|   |— Arrays / JSONB general → GIN
|   |— Full-text search → GIN (prefer) or GiST
|   |— JSONB specific field equality → Expression index
|
|— Geometric / Spatial?
|   |— Points, boxes, polygons → GiST or SP-GiST
|   |— PostGIS geometry → GiST
|   |— Range types → GiST (esp. with EXCLUDE)
|   |— IP addresses (CIDR) → SP-GiST
```

- └ No suitable standard type?
 - └ Custom operator class + GiST or SP-GiST

The 10 Commandments of Index Design

1. **Always index foreign keys** — PostgreSQL won't do it for you
2. **Use CONCURRENTLY** for all production CREATE/DROP/REINDEX operations
3. **Check `pg_stats.correlation`** before creating BRIN indexes
4. **Equality columns first** in composite indexes; range columns last
5. **Use INCLUDE** to enable index-only scans on hot-path queries
6. **Use partial indexes** for the "hot subset" of rows (status='pending', etc.)
7. **Set `random_page_cost = 1.1`** on SSD storage
8. **Never ignore `idx_scan = 0`** — remove unused indexes
9. **Run ANALYZE** after bulk loads and major data changes
10. **Tune autovacuum** aggressively for tables with heavy UPDATE/DELETE

When Indexes Are NOT the Answer

- Table has fewer than 1,000 rows → sequential scan is fast enough
- Query returns > 20% of rows → sequential scan likely cheaper
- Data is not physically correlated and BRIN was considered → use B-tree
- The bottleneck is CPU (complex functions), not I/O → optimize the function
- The query is a reporting/analytics query touching most rows → consider materialized views or pre-aggregation
- The real problem is N+1 queries → fix the application, not the indexes

Quick Reference: Common Index Patterns

```
-- Pattern 1: Foreign key (ALWAYS add this)
CREATE INDEX CONCURRENTLY ON orders(customer_id);

-- Pattern 2: Soft delete (exclude deleted rows)
CREATE INDEX CONCURRENTLY ON users(email) WHERE deleted_at IS NULL;
CREATE UNIQUE INDEX CONCURRENTLY ON users(email) WHERE deleted_at IS NULL;

-- Pattern 3: Queue/processing (hot subset)
CREATE INDEX CONCURRENTLY ON tasks(priority DESC, created_at)
WHERE status = 'pending';

-- Pattern 4: Case-insensitive lookup
CREATE INDEX CONCURRENTLY ON users(lower(email));
CREATE UNIQUE INDEX CONCURRENTLY ON users(lower(email));

-- Pattern 5: API query (composite + covering + sorted)
CREATE INDEX CONCURRENTLY ON orders(tenant_id, status, created_at DESC)
INCLUDE (id, total_amount);

-- Pattern 6: JSONB field lookup
CREATE INDEX CONCURRENTLY ON events((data->>'user_id'));
CREATE INDEX CONCURRENTLY ON events USING GIN (data);
```

```

-- Pattern 7: Full-text search
ALTER TABLE articles ADD COLUMN search_vec TSVECTOR
    GENERATED ALWAYS AS (to_tsvector('english', title || ' ' || body)) STORED;
CREATE INDEX CONCURRENTLY ON articles USING GIN (search_vec);

-- Pattern 8: Array containment
CREATE INDEX CONCURRENTLY ON posts USING GIN (tags);

-- Pattern 9: Range/booking overlap prevention
CREATE INDEX CONCURRENTLY ON reservations USING GIST (room_id, period);
ALTER TABLE reservations ADD CONSTRAINT no_overlap
    EXCLUDE USING GIST (room_id WITH =, period WITH &&);

-- Pattern 10: Time-series large table (BRIN)
CREATE INDEX CONCURRENTLY ON events USING BRIN (created_at)
WITH (autosummarize = on);

```

Monitoring Queries (Save These)

```

-- Unused indexes (review for removal)
SELECT schemaname, relname AS table, indexrelname AS index,
    idx_scan, pg_size_pretty(pg_relation_size(indexrelid)) AS size
FROM pg_stat_user_indexes
JOIN pg_index USING (indexrelid)
WHERE idx_scan = 0 AND NOT indisunique AND NOT indisprimary
ORDER BY pg_relation_size(indexrelid) DESC;

-- Index health overview
SELECT relname, indexrelname, idx_scan,
    pg_size_pretty(pg_relation_size(indexrelid)) AS idx_size,
    CASE WHEN NOT indisvalid THEN 'INVALID' ELSE 'OK' END AS status
FROM pg_stat_user_indexes JOIN pg_index USING (indexrelid)
ORDER BY pg_relation_size(indexrelid) DESC;

-- Tables needing vacuum (dead tuple accumulation)
SELECT relname, n_dead_tup, n_live_tup, last_autovacuum
FROM pg_stat_user_tables
WHERE n_dead_tup > 10000
ORDER BY n_dead_tup DESC;

-- Check bloat (requires pgstattuple extension)
-- CREATE EXTENSION pgstattuple;
-- SELECT * FROM pgstatindex('index_name');

```

Cross-Module References

Topic	File
-------	------

EXPLAIN output reading	../08 Query Optimization/01 explain basics.md
EXPLAIN ANALYZE deep dive	../08 Query Optimization/02 explain analyze.md
Scan type selection	../08 Query Optimization/03 scan types.md
Planner statistics	../08 Query Optimization/05 query planner.md
Statistics and ANALYZE	../08 Query Optimization/06 statistics and estimates.md
VACUUM internals	../10 PostgreSQL Internals/
Table partitioning	../05 PostgreSQL Core/